ETL - Data Engineering

Part 1 — Source Discovery

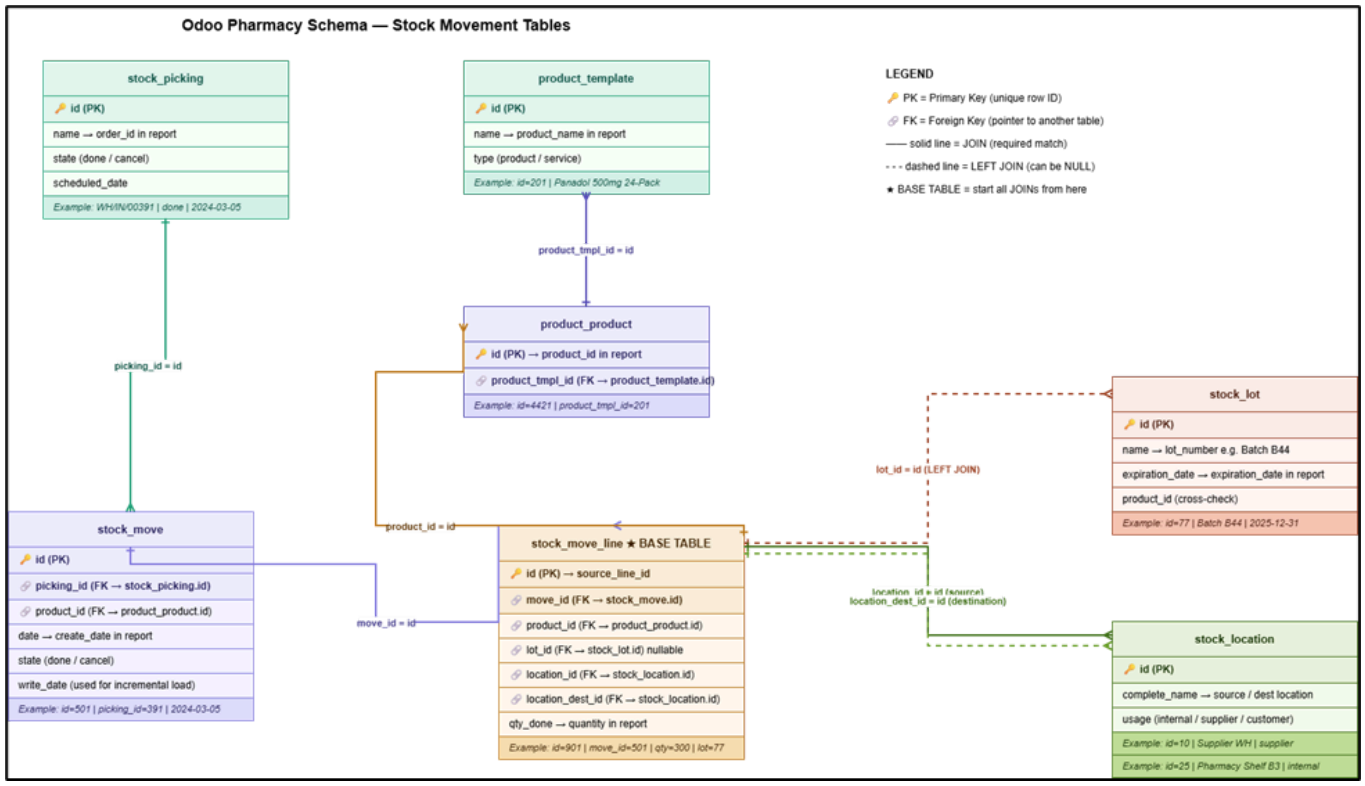
1.1 Source System Overview

The source system is Odoo ERP running on PostgreSQL. Each pharmacy client operates as an isolated tenant with its own dedicated database. The target dataset covers stock movement outcomes across all tenants.

1.2 Required Tables

Table	Role
`stock_move_line`	Base table — actual quantity moved per batch per product
`stock_move`	Movement date and status — links to order
`stock_picking`	The order reference — source of `order_id`
`stock_lot`	Batch and lot details — source of `expiration_date`
`product_product`	Product variant — source of `product_id`
`product_template`	Product master — source of product name
`stock_location`	Warehouse locations — joined twice for source and destination

> See attached schema diagram for full table structure, column definitions, and join relationships.



1.3 Join Strategy

The extraction query starts from `stock_move_line` as the base table and joins upward. `stock_lot` uses a LEFT JOIN because not all products are lot-tracked — missing lots produce a NULL `expiration_date` which is expected and flagged in the transformation layer.

`stock_move_line.product_id` is used over `stock_move.product_id` because it reflects what was physically scanned — not what was planned. In pharmacy environments substitutions occur.

1.4 Extraction Query

sql

```
SELECT
    sp.name                 AS order_id,
    pp.id                   AS product_id,
    sm.date                 AS create_date,
    sml.qty_done            AS quantity,
    sl.expiration_date      AS expiration_date,
    pt.name                 AS product_name,
    sl.name                 AS lot_number,
    src.complete_name       AS source_location,
    dst.complete_name       AS destination_location,
    sml.id                  AS source_line_id,
    sm.write_date           AS source_last_updated
```

```sql
FROM      stock_move_line   sml
JOIN      stock_move        sm   ON sm.id  = sml.move_id
JOIN      stock_picking     sp   ON sp.id  = sm.picking_id
LEFT JOIN stock_lot         sl   ON sl.id  = sml.lot_id
JOIN      product_product   pp   ON pp.id  = sml.product_id
JOIN      product_template  pt   ON pt.id  = pp.product_tmpl_id
JOIN      stock_location    src  ON src.id = sml.location_id
JOIN      stock_location    dst  ON dst.id = sml.location_dest_id

WHERE sm.state = 'done'
ORDER BY sm.date DESC;
```

1.5 Source Model Validation

Before trusting the source, the following checks run against each tenant database:

sql

```sql
-- Check 1: confirm all 7 tables exist
SELECT table_name FROM information_schema.tables
WHERE table_schema = 'public'
AND table_name IN (
  'stock_move_line','stock_move','stock_picking',
  'stock_lot','product_product','product_template','stock_location'
);

-- Check 2: row count sanity — move_lines must exceed moves
SELECT
  (SELECT COUNT(*) FROM stock_move       WHERE state='done') AS moves,
  (SELECT COUNT(*) FROM stock_move_line)                     AS move_lines;

-- Check 3: confirm LEFT JOIN is needed
SELECT COUNT(*) AS lines_without_lot
FROM stock_move_line WHERE lot_id IS NULL;

-- Check 4: confirm Odoo version (affects table name)
SELECT table_name FROM information_schema.tables
WHERE table_name IN ('stock_lot','stock_production_lot');
-- stock_lot = v16+   |   stock_production_lot = v15 and below
```

1.6 Schema Notes

The Odoo source follows a **snowflake schema** — product information is split across `product_product` and `product_template`, requiring two JOIN hops to retrieve a name. The

ETL flattens this into a **wide denormalised table** in ClickHouse eliminating all JOIN requirements at query time.

Part 2 — Extraction Design

2.1 Challenge

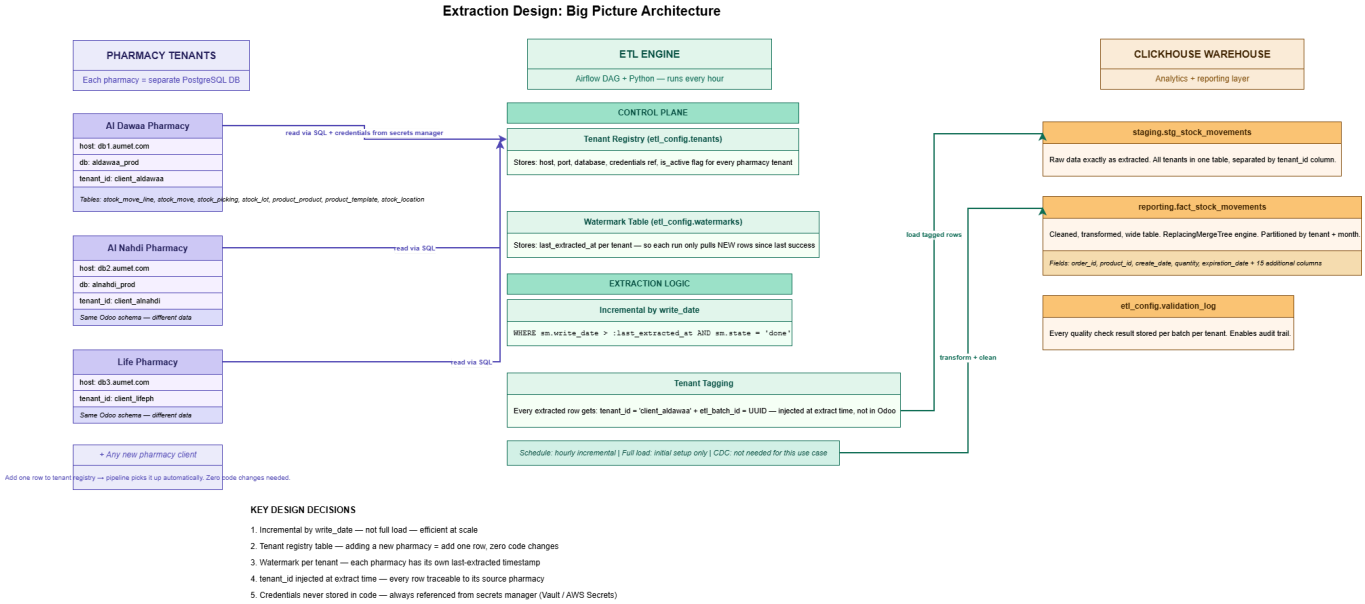
Each pharmacy client runs on a separate PostgreSQL database. The ETL must connect to each tenant independently, extract only new data since the last run, tag every row with its source tenant, and load everything into a single ClickHouse warehouse without mixing data across clients.

2.2 Tenant Registry

All tenant connection details are stored in a central config table. Adding a new pharmacy client requires no code changes — one new row is sufficient.

sql

```sql
CREATE TABLE etl_config.tenants (
    tenant_id        VARCHAR       PRIMARY KEY,
    tenant_name      VARCHAR,
    pg_host          VARCHAR,
    pg_port          INTEGER,
    pg_database      VARCHAR,
    pg_user          VARCHAR,
    pg_password_ref  VARCHAR,  -- reference to secrets manager only
    is_active        BOOLEAN   DEFAULT TRUE,
    last_synced_at   TIMESTAMP
);
```



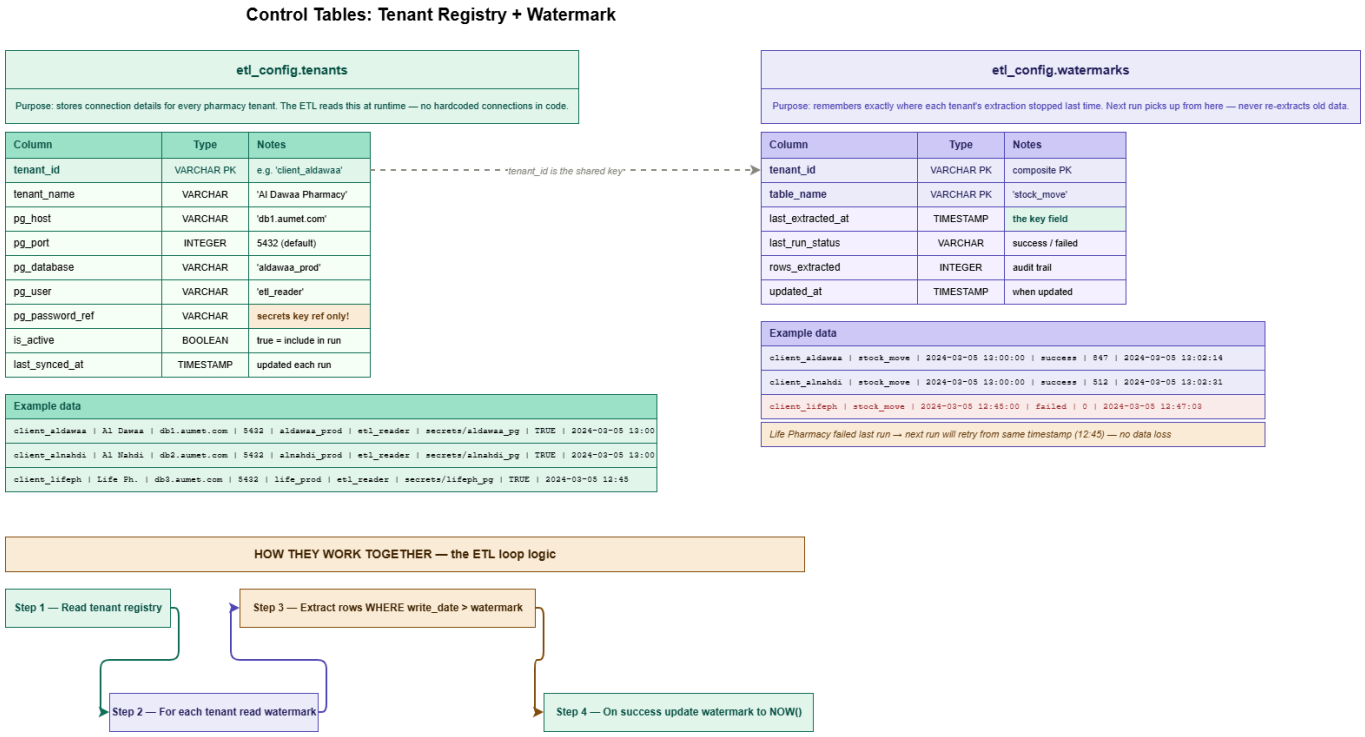
2.3 Watermark Table

Each tenant maintains its own extraction timestamp. The watermark only updates on a successful run — failed runs automatically retry from the same point on the next execution.

sql

```sql
CREATE TABLE etl_config.watermarks (
    tenant_id          VARCHAR,
    table_name         VARCHAR,
    last_extracted_at  TIMESTAMP,
    last_run_status    VARCHAR,   -- 'success' / 'failed'
    rows_extracted     INTEGER,
    updated_at         TIMESTAMP,
    PRIMARY KEY (tenant_id, table_name)
);
```

2.4 Extraction Approach



Approach	Decision
Full load	Initial setup and backfills only
Incremental by `write_date`	Primary approach — pulls only rows modified since last run
CDC	Not required — hourly frequency sufficient for pharmacy reporting

Incremental filter applied to every extraction run:

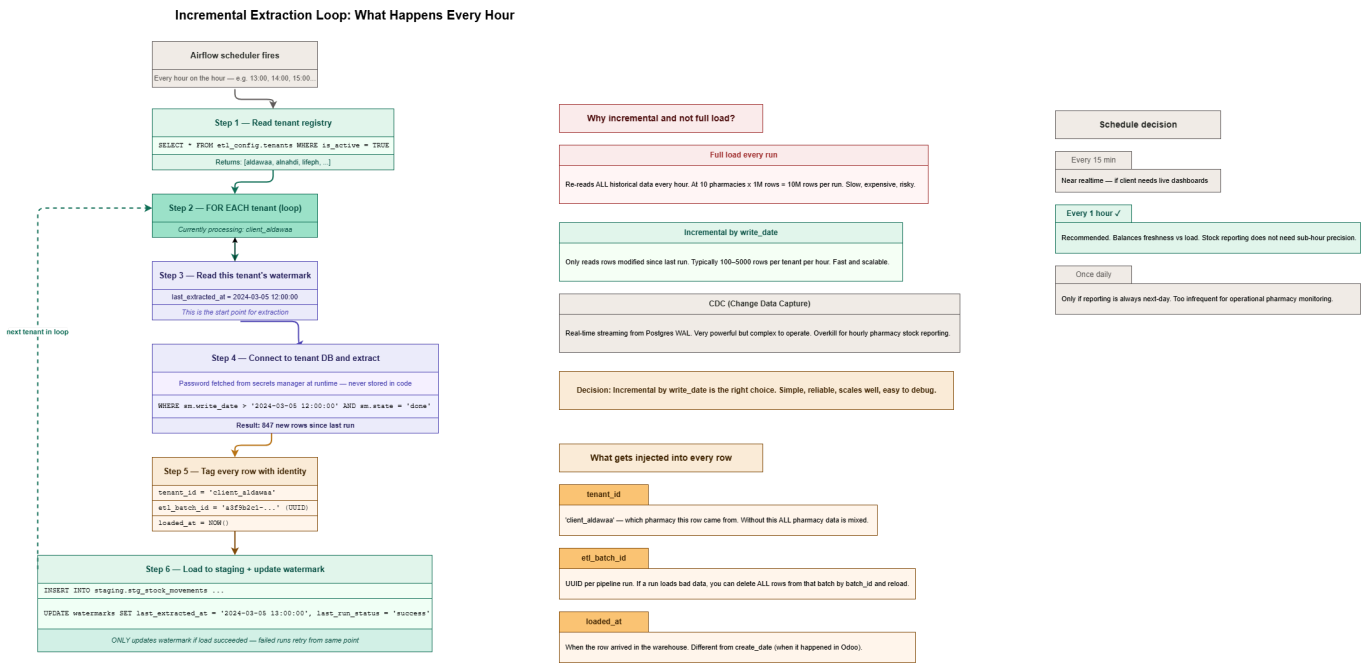
sql

```
WHERE sm.write_date > :last_extracted_at
  AND sm.state = 'done'
```

2.5 Tenant Tagging

Two fields are injected at extraction time into every row — neither exists in Odoo:

Field	Value	Purpose
`tenant_id`	e.g. `client_aldawaa`	Identifies which pharmacy the row came from
`etl_batch_id`	UUID per run	Enables surgical rollback of any bad batch



2.6 Schedule

Hourly incremental extraction. Full load on initial onboarding only.

> See attached architecture diagram for the complete end-to-end flow including the tenant loop, watermark update cycle, and ClickHouse load pattern.

Part 3 — Transformation Logic

3.1 What is Already Covered

The extraction SQL (Part 1) derives all 5 required fields and joins all 6 source tables into one flat row. The two-layer load pattern — staging then fact — is visible in the Task 2 architecture diagram showing raw rows landing in `staging.stg_stock_movements` before moving to `reporting.fact_stock_movements`. Duplicate handling via `ReplacingMergeTree` and the full ClickHouse DDL are defined in Part 4.

3.2 Two-Layer Design

Raw data lands in `staging.stg_stock_movements` untouched. If transformation fails, raw data is preserved and reprocessed without re-extracting from source. The fact table only receives clean validated rows.

3.3 Cleaning Rules Applied in Transformation

These rules run when moving staging → fact:

Field	Rule
`expiration_date` NULL	Keep NULL — flag `is_expiration_missing = TRUE`
`quantity` negative	ABS(value) — flag `is_quantity_negative = TRUE`
`quantity` NULL	Default to 0
`product_name` blank	Replace with `UNKNOWN`
`order_id` NULL	Reject row
`product_id` NULL	Reject row
`lot_number` NULL	Allow — not all products are lot-tracked

3.4 Business Rule Ambiguity

When source behaviour is unclear the rule is: **keep raw data honest, flag for review, never silently default.**

Three decisions made under ambiguity:

- NULL `expiration_date` is kept as NULL — never defaulted to a fake date — because it could mean the product genuinely has no expiry or that Odoo was not configured correctly. The analyst decides.
- Negative quantities are corrected to absolute value but always flagged — because they could be legitimate reversal entries or data errors.
- When `stock_move.product_id` and `stock_move_line.product_id` differ, `stock_move_line` always wins — it reflects what was physically scanned not what was planned.

Part 4 — Warehouse Modeling

4.1 What is Already Covered

- Full ClickHouse DDL including engine, partitioning, ordering, and all columns is defined in the Step 4 SQL
- Star vs snowflake vs wide table decision and reasoning is covered in the schema diagram section

4.2 Model Choice — Wide Denormalised Table

- ClickHouse is a columnar analytics engine — JOINs are expensive and slow down queries at scale
- Star and snowflake schemas require JOINs at query time — not suitable for this engine
- Wide table absorbs all JOIN complexity at extraction time — analysts never write JOINs
- ETL runs the 6-table JOIN once per hour and writes one flat row per movement to ClickHouse
- Result: any report is a simple SELECT with WHERE filters — no Odoo knowledge required

4.3 Table Grain

- One row per batch per product per movement — driven by `stock_move_line`
- If one order has Panadol in two batches → two rows — because each has a different lot and expiry date
- This grain ensures no expiry information is ever lost through aggregation
- Natural unique key: `(tenant_id, source_line_id)` — globally unique across all pharmacy tenants

4.4 Partitioning and Ordering

Partitioning — `PARTITION BY (tenant_id, toYYYYMM(create_date))`

- Each pharmacy gets its own monthly partitions
- A query filtering by `tenant_id` and date range skips every other pharmacy's partitions entirely
- ClickHouse never reads data it does not need

Ordering — `ORDER BY (tenant_id, create_date, order_id, product_id, source_line_id)`

- Data is physically sorted on disk to match the most common query patterns
- Tenant + date range filters — which cover almost every report — hit the right data blocks immediately
- No full table scan needed for standard analytical queries

4.5 How the Model Supports Future Reporting

Future requirement	How it is supported
New pharmacy tenant onboarded	`tenant_id` on every row — zero schema changes
Expiry risk dashboard	`expiration_date` + `is_expiration_missing` already present

Future requirement	How it is supported
Lot traceability or product recall	`lot_number` + `source_line_id` enable full trace
Inbound vs outbound volume	`movement_direction` already derived
Rollback a bad ETL batch	Delete `WHERE etl_batch_id = 'x'` and reload
New reporting columns needed	Wide table — add column without breaking existing queries

Part 5 — Data Quality and Validation

5.1 What is Already Covered

- All validation SQL — row counts, reconciliation, duplicate detection, business rule checks, and tenant silence detection — is defined in the Step 5 SQL
- The `etl_config.validation_log` table DDL and purpose is defined in Step 5
- The three-layer validation approach is shown in the Step 5 section

5.2 Validation Approach

Three layers run after every batch load:

- **Layer 1 — Row count** — source row count vs target row count per tenant per batch. Mismatch blocks watermark update and forces retry
- **Layer 2 — Reconciliation** — total quantity per order compared between PostgreSQL source and ClickHouse target. Catches transformation errors that silently change values
- **Layer 3 — Business rules** — duplicate detection, missing expiration spike, invalid product mappings, negative quantities, expiry before movement date, and tenant silence check

5.3 Validation Result Tracking

- Every check result is written to `etl_config.validation_log` with `check_name`, `check_status`, `source_value`, `target_value`, and `etl_batch_id`
- Full audit trail — every batch is traceable, every failure is queryable
- If a batch fails any Layer 1 or Layer 2 check the watermark is not updated — the next run retries from the same point

5.4 Checks Summary

Check	Layer	What it catches
Row count match	1	Dropped rows during extraction or load
Quantity reconciliation	2	Transformation errors changing values

Check	Layer	What it catches
Duplicate detection	3	Re-insertion bugs
Missing expiration spike	3	Source data quality degradation
Invalid product or order	3	Bad joins or rejected rows accumulating
Expiry before create_date	3	Upstream data entry errors in Odoo
Tenant silence check	3	Pipeline failing silently for a tenant

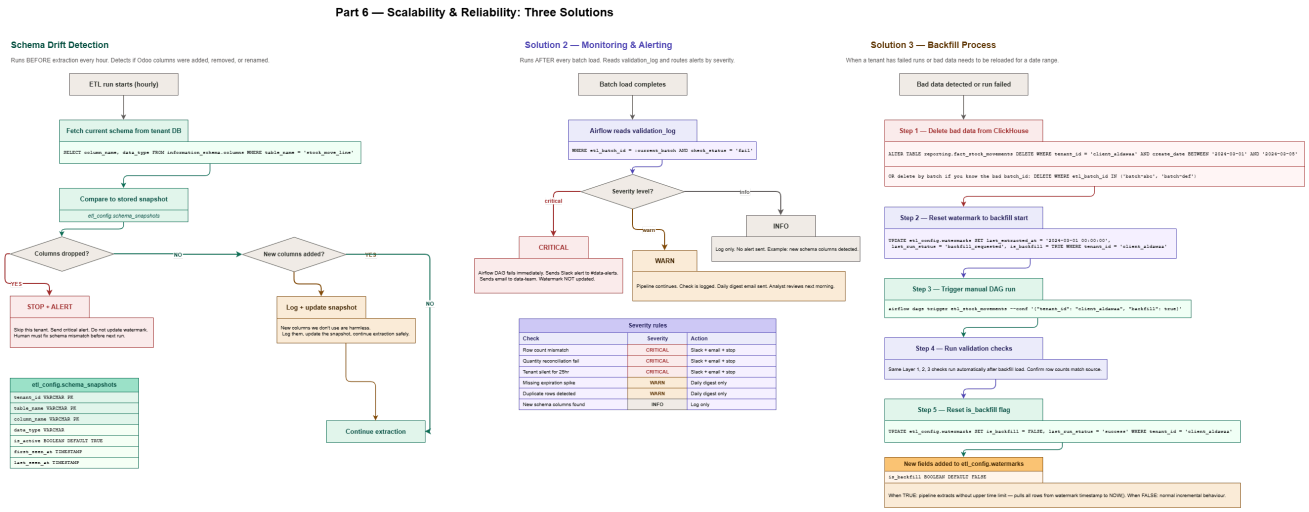
Part 6 — Scalability and Reliability

6.1 What is Already Covered

- Tenant growth is handled by the tenant registry pattern — adding a new pharmacy requires one row, zero code changes. Covered in Task 2 architecture diagram.
- Data volume growth is handled by incremental extraction — only new rows per run, never full reload. Covered in Task 2 watermark design.
- Failed runs and retries are handled by the watermark — only updates on success, failed runs retry from the same point automatically. Covered in Task 2 watermark diagram.
- New reporting requirements are handled by the wide table design — new columns can be added without breaking existing queries. Covered in Part 4.

6.2 Schema Drift

> See Part 6 diagram — Solution 1: Schema Drift Detection.



Before every extraction run a schema fingerprint check runs against each tenant database. It compares the live schema to a stored snapshot in `etl_config.schema_snapshots`. Three outcomes:

- **Dropped column** — stop immediately, skip this tenant, send critical alert. A dropped column we depend on breaks extraction silently if not caught here.
- **New column added** — log it, update the snapshot, continue safely. New columns we do not use are harmless.
- **No change** — proceed directly to extraction.

This check adds one extra query per tenant per run — negligible overhead, high protection value.

6.3 Monitoring and Alerting

> See attached Part 6 diagram — Solution 2: Monitoring and Alerting.

This is a separate concern from data quality validation (Part 5). Part 5 defines what to check. Part 6 defines where alerts go and who acts on them. They are separated deliberately — validation logic should not change just because an alert destination changes.

After every batch the Airflow DAG reads `etl_config.validation_log` and routes by severity:

Severity	Trigger	Action
Critical	Row count mismatch, quantity reconciliation fail, tenant silent 25hr	DAG fails, Slack alert to `#data-alerts`, email to data team, watermark not updated
Warn	Missing expiration spike, duplicate rows	Pipeline continues, daily digest email only
Info	New schema columns detected	Log only, no alert

The `severity` field is added to `validation_log`:

sql

```sql
ALTER TABLE etl_config.validation_log
ADD COLUMN severity VARCHAR DEFAULT 'warn';
-- 'critical' / 'warn' / 'info'
```

6.4 Backfills

> See attached Part 6 diagram — Solution 3: Backfill Process.

A backfill is needed when a tenant had failed runs or bad data was loaded for a date range. The process is five steps:

- **Step 1** — Delete bad rows from ClickHouse by date range or `etl_batch_id`
- **Step 2** — Reset the watermark to the start of the affected period and set `is_backfill = TRUE`
- **Step 3** — Trigger a manual Airflow DAG run for that tenant only
- **Step 4** — Validation checks run automatically after load — confirm row counts match source
- **Step 5** — Reset `is_backfill = FALSE` and mark watermark as success

One field is added to `etl_config.watermarks`:

sql

```sql
ALTER TABLE etl_config.watermarks
ADD COLUMN is_backfill BOOLEAN DEFAULT FALSE;
-- When TRUE: extracts from watermark timestamp to NOW() with no upper limit
-- When FALSE: normal incremental behaviour
```

The backfill uses the exact same extraction loop as normal runs — no separate code path needed.

Part 7 — Documentation and Handover

7.1 What is Already Covered

- Full architecture diagram in Task 2 draw.io file covers end-to-end pipeline design
- All extraction, transformation, and validation SQL is documented in Parts 1–5
- Assumptions are listed in Part 1 section 1.6 and Part 3 section 3.6
- Schema drift, monitoring, and backfill processes are documented in Part 6 draw.io diagram

---

7.2 Glossary

Term	Plain English definition
Tenant	A single pharmacy client with its own isolated PostgreSQL database — e.g. Al Dawaa, Al Nahdi
Tenant ID	A short code that identifies which pharmacy a row came from — e.g. `client_aldawaa`
Stock movement	Any physical transfer of a product — received from supplier, sold to customer, or moved between locations

Term	Plain English definition
Movement outcome	The recorded result of a completed stock movement — quantity, product, date, lot, expiry
Lot / batch	A group of units of the same product manufactured together — identified by lot number and expiry date
Expiration date	The date a medicine lot expires — NULL means the product is not lot-tracked
Watermark	A timestamp recording when data was last successfully extracted for a tenant — used to extract only new rows on the next run
ETL batch ID	A unique ID (UUID) assigned to each pipeline run — used to trace or roll back any specific load
Incremental load	Extracting only rows modified since the last successful run — not the full history
Full load	Extracting all historical data — used only for initial setup or backfills
Backfill	Reloading data for a specific tenant and date range — used when a run fails or bad data is detected
Staging table	The raw landing zone in ClickHouse — data arrives here untouched before transformation
Fact table	The clean analytics-ready reporting table in ClickHouse — what analysts query
Snowflake schema	How Odoo stores data in PostgreSQL — normalised across many related tables requiring multiple JOINs
Wide table	How the warehouse stores data in ClickHouse — one flat row per movement, no JOINs needed
ReplacingMergeTree	A ClickHouse engine that automatically keeps the latest version of a row when duplicates arrive
Schema drift	When an Odoo database column is added, renamed, or removed — detected by the schema fingerprint check
Validation log	A table storing the result of every quality check per batch per tenant — the audit trail
Movement direction	Derived field — inbound (supplier to pharmacy), outbound (pharmacy to customer), internal (between locations)

7.3 Data Dictionary — `reporting.fact_stock_movements`

Column	Type	Source table	Meaning
`tenant_id`	String	Injected by ETL	Which pharmacy client this rov belongs to
`order_id`	String	`stock_picking.name`	Transfer reference number e.g. WH/IN/0039
`product_id`	Int64	`product_product.id`	Unique ID of the product variant
`product_name`	String	`product_template.name`	Human readable product nam e.g. Panadol 500mg
`create_date`	Date	`stock_move.date`	Date the movement w completed
`quantity`	Float64	`stock_move_line.qty_done`	Actual quant physically moved
`expiration_date`	Nullable(Date)	`stock_lot.expiration_date`	Expiry date c the lot
`lot_number`	Nullable(String)	`stock_lot.name`	Lot/batch identifier e.g. Batch B44
`source_location`	String	`stock_location.complete_name`	Where the product cam from
`destination_location`	String	`stock_location.complete_name`	Where the product went
`movement_direction`	String	Derived	inbound / outbound / internal / oth

Column	Type	Source table	Meaning
`order_status`	String	`stock_picking.state`	Status of the transfer — always done
`is_expiration_missing`	Bool	Derived	TRUE if expiration_da is NULL
`is_quantity_negative`	Bool	Derived	TRUE if original qty_done wa negative
`is_lot_missing`	Bool	Derived	TRUE if lot_number is NULL
`source_line_id`	Int64	`stock_move_line.id`	Source recor ID — traceability back to Odoo
`source_move_id`	Int64	`stock_move.id`	Parent move ID in Odoo
`source_last_updated`	DateTime	`stock_move.write_date`	When the source recor was last modified in Odoo
`etl_batch_id`	String	Injected by ETL	UUID of the pipeline run that loaded t row
`loaded_at`	DateTime	Injected by ETL	When the row arrived in the staging table
`dwh_updated_at`	DateTime	Injected by ETL	When the row was written t the fact table

---

7.4 Business Matrix

Maps business reporting needs to the fields required to answer them. Any analyst can use this as a reference without needing to understand the underlying schema.

Business question	Required fields
What stock moved for pharmacy X this month?	`tenant_id`, `create_date`, `order_id`, `product_name`, `quantity`
Which medicine lots are expiring in the next 30 days?	`expiration_date`, `lot_number`, `product_name`, `quantity`, `tenant_id`
Which products have no expiry date tracked?	`is_expiration_missing`, `product_name`, `tenant_id`
How much stock did we receive vs ship this month?	`movement_direction`, `quantity`, `create_date`, `tenant_id`
Trace a specific lot across all pharmacies	`lot_number`, `tenant_id`, `order_id`, `source_location`, `destination_location`
Compare stock movement volume across pharmacies	`tenant_id`, `quantity`, `create_date`
Investigate a suspicious movement	`source_line_id`, `etl_batch_id`, `source_last_updated`, `order_id`
Were any negative quantities recorded?	`is_quantity_negative`, `product_name`, `tenant_id`, `create_date`
Which ETL batch loaded a specific row?	`etl_batch_id`, `loaded_at`, `source_line_id`
What moved from supplier to pharmacy shelf?	`movement_direction = inbound`, `source_location`, `destination_location`

7.5 Runbook — Operational Guide

Five scenarios covering everything an on-call engineer needs:

Scenario 1 — How to check pipeline status

sql

```sql
-- Check last run per tenant
SELECT
    tenant_id,
```

```sql
    last_extracted_at,
    last_run_status,
    rows_extracted,
    updated_at
FROM etl_config.watermarks
ORDER BY updated_at DESC;

-- Check recent validation failures
SELECT tenant_id, check_name, check_status, detail_message, checked_at
FROM etl_config.validation_log
WHERE check_status = 'fail'
  AND checked_at  > now() - INTERVAL 24 HOUR
ORDER BY checked_at DESC;
```

Scenario 2 — What to do when a run fails

- Check Airflow for the failed task and read the error log
- Check `etl_config.watermarks` — if `last_run_status = 'failed'` the watermark was not updated — the next run will retry automatically from the same point
- If the failure is a connection issue — verify tenant DB credentials in secrets manager
- If the failure is a schema drift alert — check `etl_config.schema_snapshots` for what changed and update the extraction query if needed
- If the failure is a validation failure — check `etl_config.validation_log` for the specific check that failed

---

Scenario 3 — How to add a new pharmacy tenant

sql

```sql
-- Add one row to the tenant registry
INSERT INTO etl_config.tenants VALUES (
    'client_newpharmacy',
    'New Pharmacy Co.',
    'db4.aumet.com',
    5432,
    'newpharmacy_prod',
    'etl_reader',
    'secrets/newpharmacy_pg',
    TRUE,
    now()
```

```sql
);
-- No code changes needed. Pipeline picks it up on next run.
```

Scenario 4 — How to trigger a backfill

sql

```sql
-- Step 1: delete the bad data
ALTER TABLE reporting.fact_stock_movements
DELETE WHERE tenant_id   = 'client_aldawaa'
         AND create_date >= '2024-03-01'
         AND create_date <= '2024-03-05';

-- Step 2: reset the watermark
UPDATE etl_config.watermarks
SET last_extracted_at = '2024-03-01 00:00:00',
    last_run_status   = 'backfill_requested',
    is_backfill       = TRUE
WHERE tenant_id  = 'client_aldawaa'
  AND table_name = 'stock_move';
```

bash

```bash
# Step 3: trigger the DAG manually
airflow dags trigger etl_stock_movements \
  --conf '{"tenant_id": "client_aldawaa", "backfill": true}'
```

Scenario 5 — How to roll back a bad batch

sql

```sql
-- Find the bad batch ID
SELECT DISTINCT etl_batch_id, loaded_at, COUNT(*) AS rows
FROM reporting.fact_stock_movements
WHERE tenant_id = 'client_aldawaa'
GROUP BY etl_batch_id, loaded_at
ORDER BY loaded_at DESC
LIMIT 10;

-- Delete all rows from that batch
```

```sql
ALTER TABLE reporting.fact_stock_movements
DELETE WHERE etl_batch_id = 'the-bad-batch-uuid-here';

-- Reset watermark to before that batch ran
UPDATE etl_config.watermarks
SET last_extracted_at = '2024-03-05 12:00:00',
    last_run_status   = 'rollback'
WHERE tenant_id = 'client_aldawaa';
```